

Inteligencia de Negocios

Willy Alexander Tapia Oré

c26096@utp.edu.pe

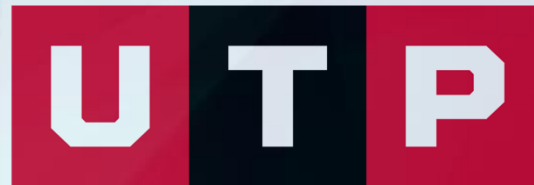
Ingeniería de Sistemas e Informática



Universidad
Tecnológica
del Perú

Big Data

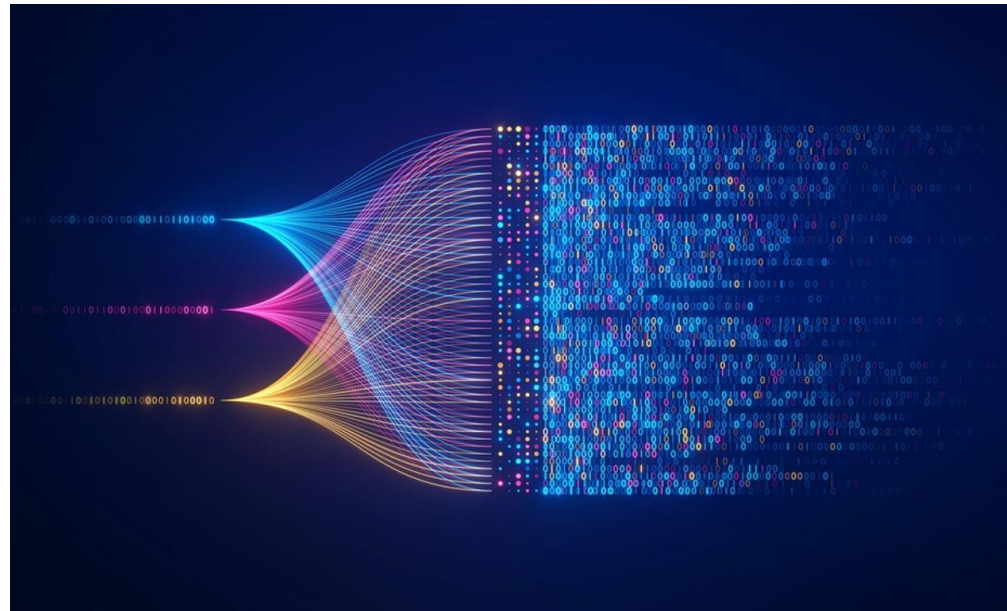
Semana 11 – Sesión 02



Universidad
Tecnológica
del Perú

Recordando la sesión anterior

¿Cuáles fueron las 3 V iniciales con las que Doug Laney definió el concepto de Big Data a principios de los 2000?



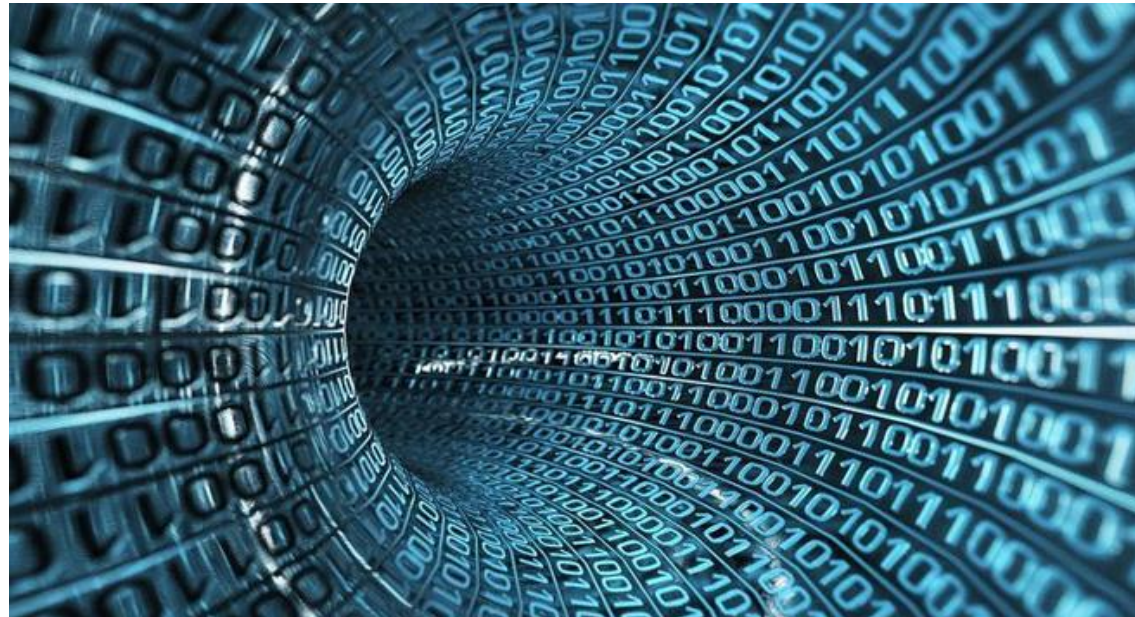
Logro de aprendizaje de la sesión

Al finalizar la sesión, el estudiante identifica la arquitectura y procesamiento de Big Data, así como el impacto del Open Data en el análisis de negocios.



Utilidad

De las 5 Vs analizadas en la sesión anterior, ¿cuál de ellas creen que representa el mayor reto para el arquitecto que diseña el sistema de almacenamiento?



Utilidad

Big Data optimiza la toma de decisiones científicas y reduce costos operativos mediante el análisis de patrones ocultos. Por su parte, los Datos Abiertos democratizan el mercado, permitiendo que las pymes accedan a inteligencia estratégica gratuita para competir con grandes corporaciones.



contenido

1

Concepto de Big data.

2

Aplicaciones prácticas de Big data en diferentes industrias.

3

Arquitectura y procesamiento de Big data

4

Open data y su impacto en el análisis de negocios.

contenido

1

Tema 1

2

Tema 2

3

Arquitectura y procesamiento de Big data

4

Práctica

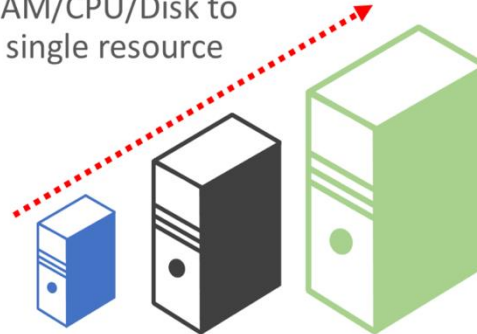
El Propósito de Organizar los Datos

La gestión de volúmenes a escala de petabytes ha forzado la transición de servidores tradicionales de escalado vertical hacia entornos distribuidos basados en arquitecturas de código abierto.

Nombre	Símbolo	Potencias binarias y valores decimales
byte	b	$2^0 = 1$
Kbyte	KB	$2^{10} = 1\,024$
Megabyte	MB	$2^{20} = 1\,048\,576$
Gigabyte	GB	$2^{30} = 1\,073\,741\,824$
Terabyte	TB	$2^{40} = 1\,099\,511\,627\,776$
Petabyte	PB	$2^{50} = 1\,125\,899\,906\,842\,624$
Exabyte	EB	$2^{60} = 1\,152\,921\,504\,606\,846\,976$
Zettabyte	ZB	$2^{70} = 1\,180\,591\,620\,717\,411\,303\,424$
Yottabyte	YB	$2^{80} = 1\,208\,925\,819\,614\,629\,174\,706\,176$

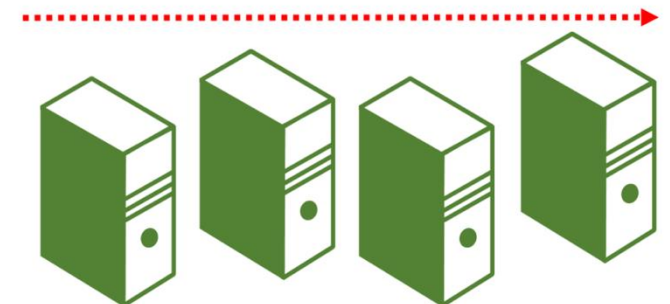
Scale Up (vertical scaling)

Increase capacity by adding RAM/CPU/Disk to a single resource



Scale Out (horizontal scaling)

Increase capacity by adding resources



Ejemplo: MegaDelivery

La empresa cuenta una plataforma de comercio electrónico y entregas a nivel nacional. Cada segundo, miles de usuarios abren la app, buscan productos, hacen clics, agregan artículos al carrito y realizan pagos. Toda esa actividad genera un tsunami de eventos que debemos capturar y procesar.

Volumen:

Almacenamos y procesamos 1 Petabyte al mes en JSONs crudos de interacciones y clics distribuidos en múltiples servidores.

Velocidad:

Los datos se transmiten de forma continua (streaming) para calcular tendencias de productos al instante y actualizar dashboards en milisegundos.

Variedad:

Gestionamos datos estructurados (tablas de usuarios/pagos), semiestructurados (archivos JSON de clics) y no estructurados (imágenes).

Veracidad:

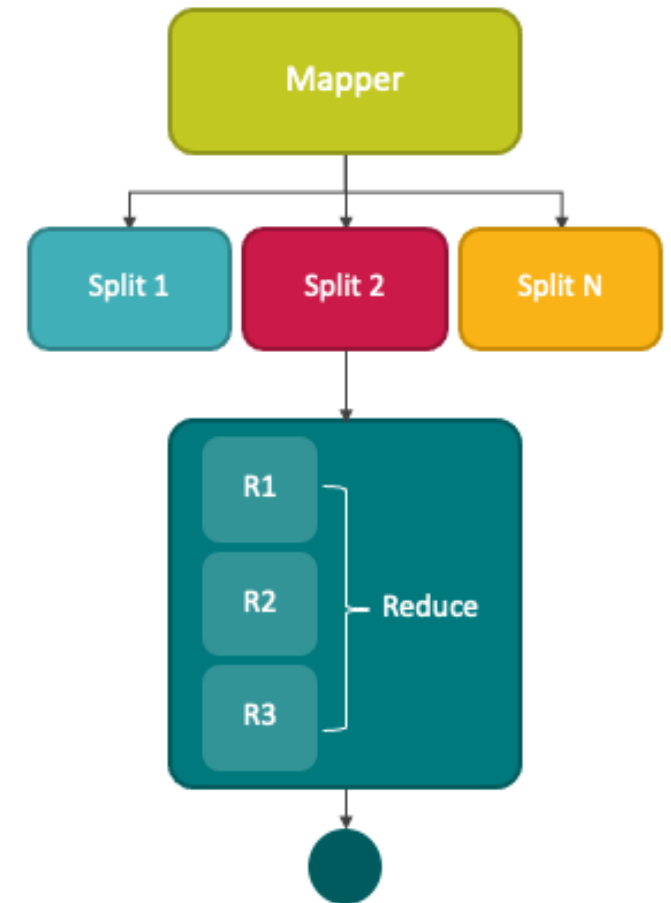
Eliminar clics fantasmas por errores de red y validar que los IDs de usuarios sean reales.

Valor:

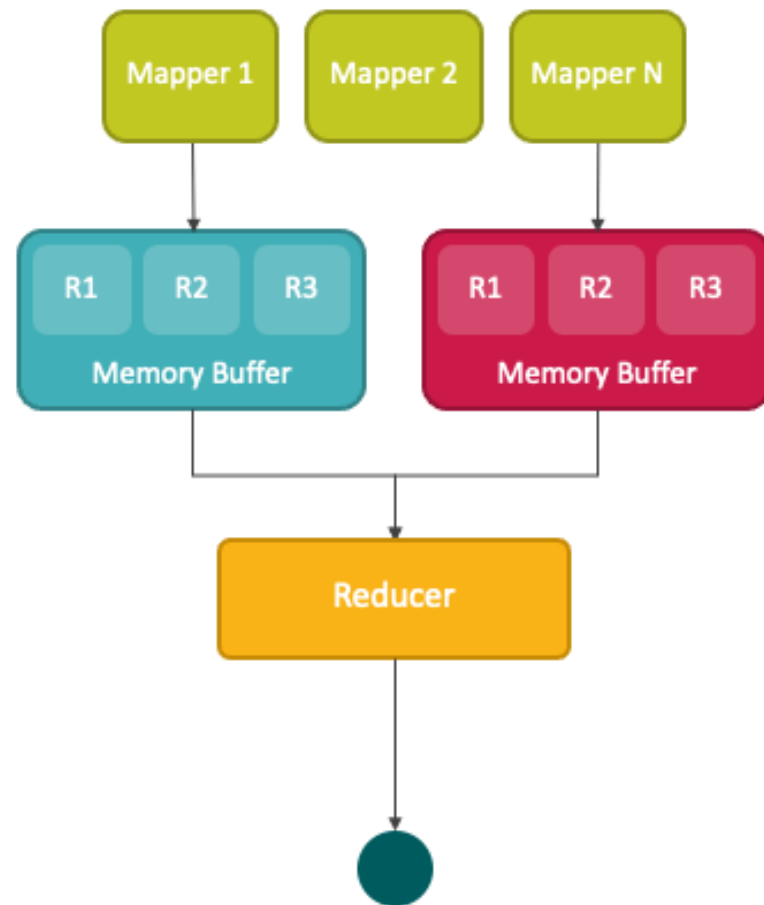
Transformamos los datos limpios para calcular el KPI de "Productos más Tendencia" y lanzar promociones comerciales dinámicas.

Ecosistemas de Big Data: Almacenamiento Distribuido y Procesamiento Masivo

Apache Hadoop: Es la base histórica que utiliza el Hadoop Distributed File System (HDFS) para almacenamiento masivo en hardware comercial y MapReduce para el procesamiento por lotes. Su fortaleza es la replicación física de datos, lo que garantiza tolerancia a fallos, aunque su dependencia de escrituras en disco lo hace más lento para análisis interactivos.



Ecosistemas de Big Data: Almacenamiento Distribuido y Procesamiento Masivo



Apache Spark: Surge como un motor analítico optimizado para el procesamiento en memoria RAM, lo que lo hace hasta 100 veces más veloz que Hadoop para ciertas cargas.

Utiliza RDD (Resilient Distributed Datasets) y un Grafo Acíclico Dirigido (DAG) para trazar transformaciones lógicas antes de la ejecución física.

Ejemplo de E-commerce: Hadoop

Escenario: Necesitamos asegurarnos de que ningún JSON de compra o clic se pierda si un servidor del clúster sufre un apagón o una falla de hardware.

Cuando los archivos JSON de nuestra app entran al sistema, los guardamos en HDFS. HDFS toma esos archivos, los fragmenta y los replica automáticamente en tres servidores distintos del clúster. Si el servidor A se quema, los datos de las transacciones siguen a salvo en los servidores B y C.

Si queremos calcular el reporte consolidado de ventas del mes, usamos MapReduce el proceso se divide (Split) entre todas las máquinas para que busquen los datos localmente en sus discos, y luego se consolidan (Reduce) los resultados. ¿El problema? Como lee y escribe constantemente en los discos duros físicos, tardará horas en darnos el reporte.

Ejemplo de E-commerce: Spark

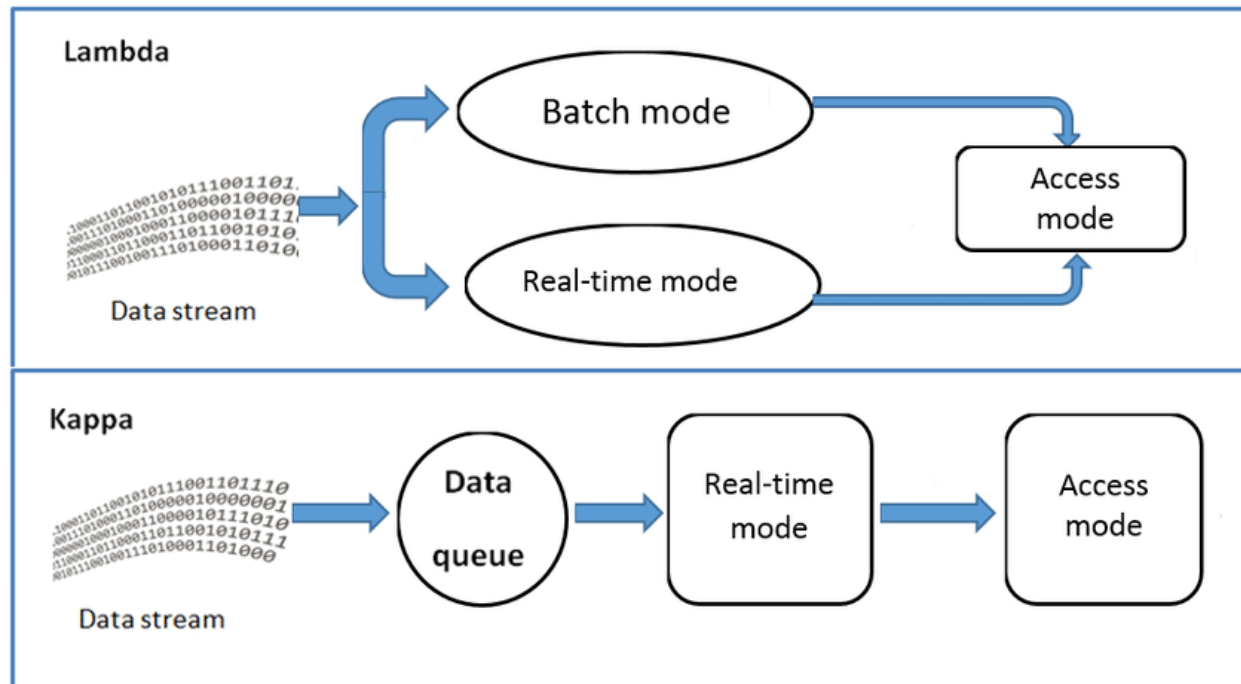
Escenario: El gerente de marketing quiere ver un panel interactivo que muestre cuáles son las categorías de productos más buscadas en este momento para lanzar cupones de descuento dinámicos. No puede esperar a que el proceso en disco de Hadoop termine mañana.

Para resolver la lentitud de Hadoop en consultas interactivas, introducimos Apache Spark. En lugar de obligar al sistema a leer el disco duro para cada consulta, Spark levanta los JSON de los clics y los mantiene cargados directamente en la memoria RAM del clúster en forma de RDD (Datasets Distribuidos Resilientes).

Si el gerente pide el KPI de 'Productos en Tendencia', Spark diseña un mapa lógico eficiente en la mente del clúster (DAG) y calcula las agregaciones en la RAM de forma inmediata. Esto hace que el procesamiento sea hasta 100 veces más rápido, permitiendo que marketing tome decisiones basadas en lo que está ocurriendo en la app prácticamente al instante."

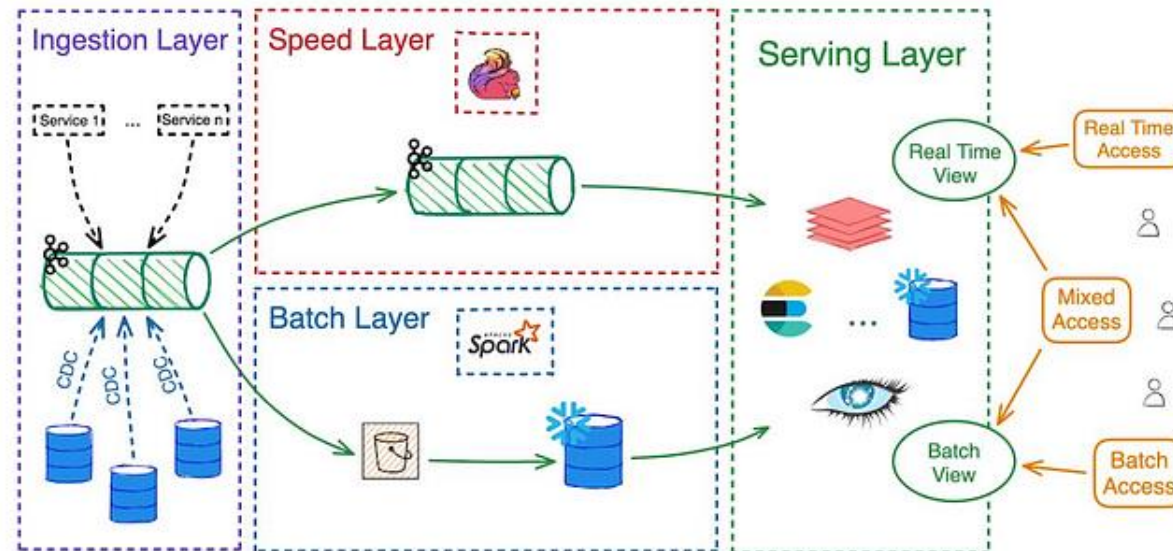
Flujos de datos (Pipelines)

Para unificar el análisis de datos históricos y en tiempo real, las empresas implementan 2 arquitecturas:



Arquitectura Lambda

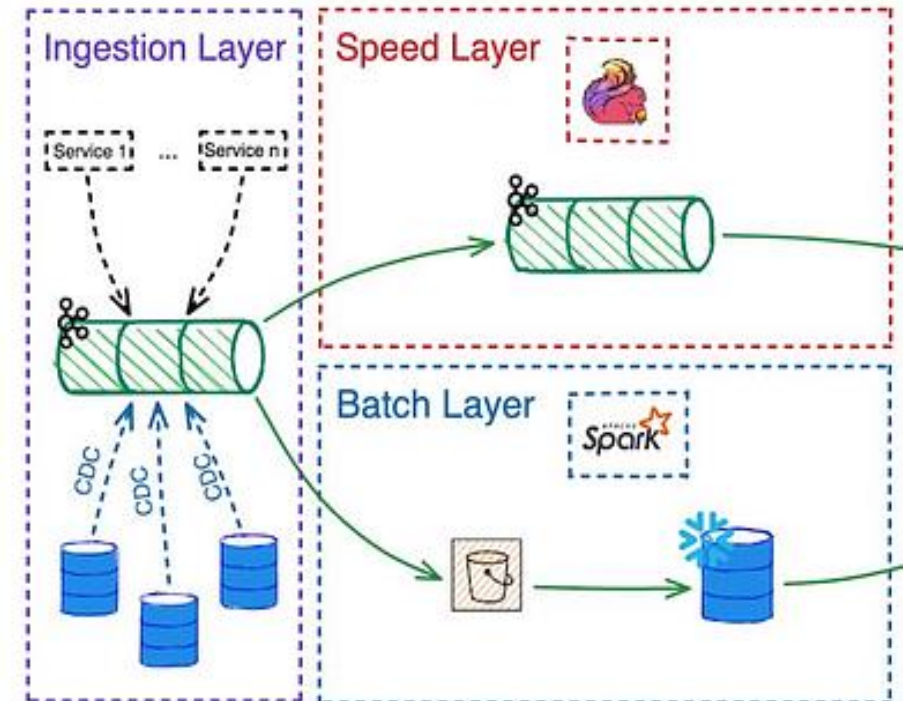
Utiliza un doble pipeline con una capa de lotes (precisión) y una capa de velocidad (baja latencia). Su desventaja es la alta complejidad de mantenimiento al duplicar la lógica de negocio.



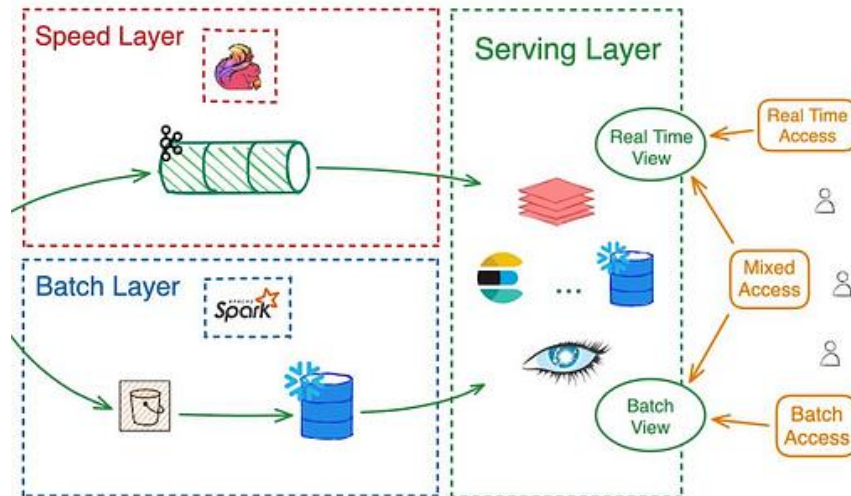
Ejemplo con arquitectura Lambda

1. Ingestion Layer: Cada vez que un usuario hace clic o compra en la app, se genera un evento. Este entra a un bus de mensajería. En Lambda, este componente está configurado para duplicar obligatoriamente cada dato recibido hacia dos tuberías independientes.

2. Batch Layer: El primer camino es el inferior. El JSON se almacena en un Data Lake en crudo. A la medianoche, Apache Spark arranca un proceso batch que toma todos los millones de JSON acumulados en el día, elimina transacciones duplicadas por caídas de red y guarda la información limpia en el storage histórico. Es lenta, pero es la verdad absoluta para el reporte.



Ejemplo con arquitectura Lambda

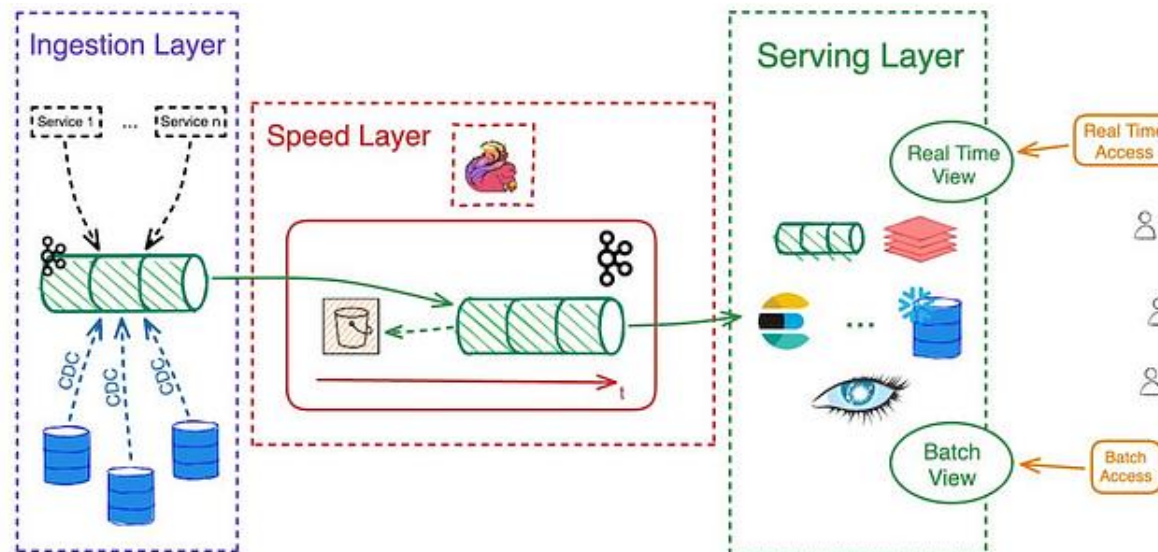


3. Speed Layer: El segundo camino es el superior. Al mismo tiempo, el JSON entra a Apache. A Flink no le importa el pasado; solo procesa los datos calientes de los últimos 10 segundos en memoria RAM para detectar si un usuario está haciendo 50 clics de compra por segundo (comportamiento de bot o fraude), actualizando la Real Time View al instante.

4. Serving Layer: Cuando el gerente abre el dashboard del e-commerce, la capa de servicio junta el reporte consolidado e inmutable de ayer con las métricas volátiles de lo que va del día de hoy.

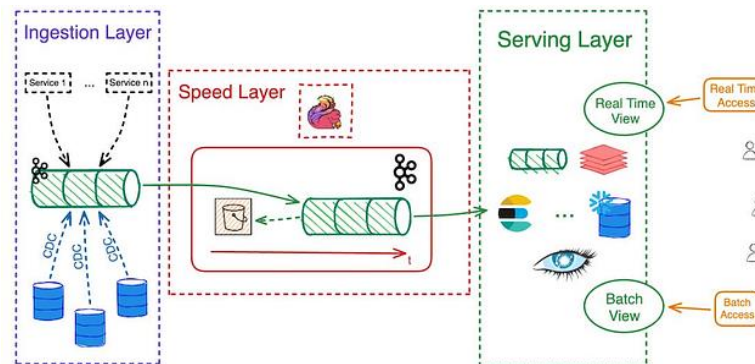
Arquitectura Kappa

Simplifica el flujo utilizando una única tecnología de transmisión continua (streaming) como Apache Kafka. Los datos se tratan como un registro inmutable que puede "rebobinarse" para recalcular históricos.



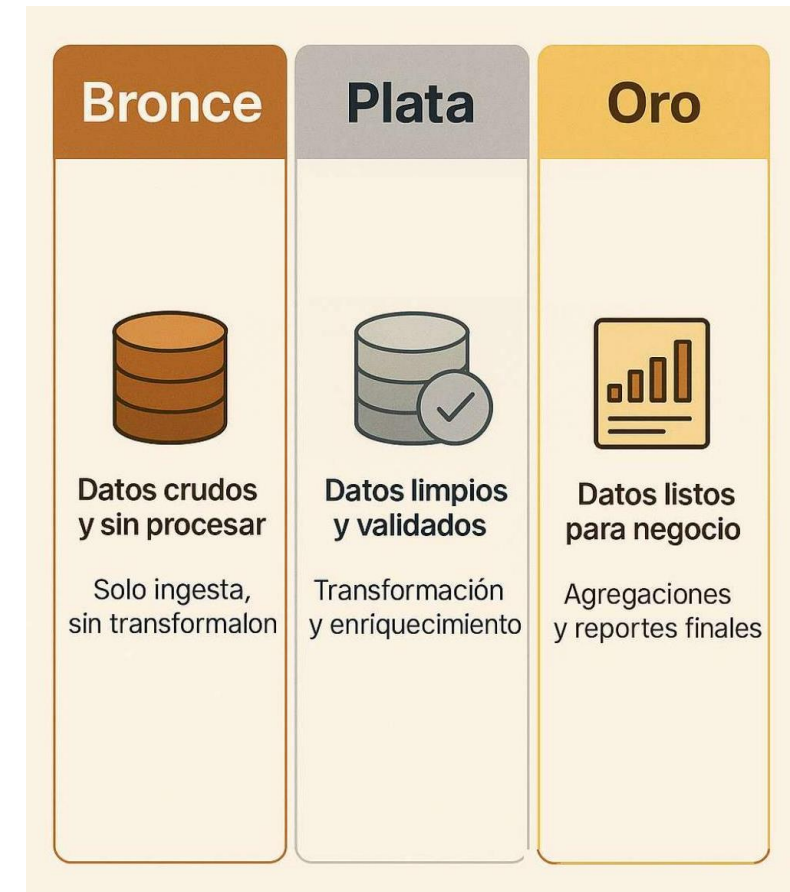
Ejemplo con arquitectura Kappa

- 1. Ingestion Layer:** El flujo de clics entra a un clúster. Pero a diferencia de Lambda, Kafka aquí no solo funciona como un puente temporal, sino como una **cola inmutable y persistente**. Guarda el historial exacto de todos los clics ordenados por el tiempo en que sucedieron.
- 2. Speed Layer:** Solo tenemos un motor de streaming (Flink) conectado permanentemente a la cola de Kafka. Este único motor procesa las alertas de fraude en tiempo real y, al mismo tiempo, va acumulando los datos para las vistas históricas. Un solo código hace todo.
- 3. Serving Layer:** Cuando el gerente de MegaDelivery abre su dashboard, la Serving Layer solo hace una consulta directa a esa única tabla centralizada. No hay cruces de datos complejos ni desfases de tiempo; el dashboard consume una única versión de la verdad que está actualizada hasta el último milisegundo.



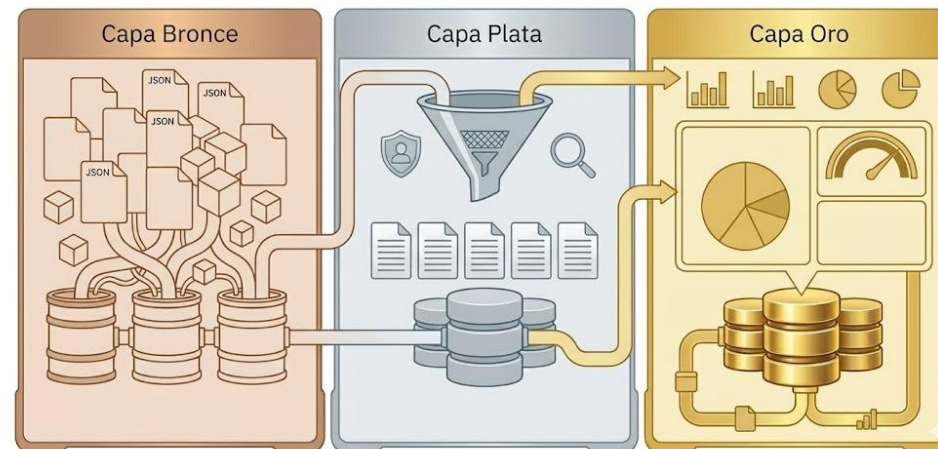
Organización de calidad - Medallion

- **Capa Bronze:** Es la fase de ingesta continua. Aquí se almacenan los datos tal cual vienen de la fuente en un almacén de datos crudos.
- **Capa Silver:** Es la fase de limpieza y estructuración. Se aplican reglas de calidad para descartar errores, se parsean los campos y se integra la información de distintas fuentes.
- **Capa Gold:** Es la fase de agregación final. Se calculan los Indicadores Clave de Rendimiento (KPIs) en tiempo real para que el gerente pueda visualizarlos en un dashboard y tomar decisiones basadas en evidencia.



Ejemplo: E-commerce

- **Bronce:** Guardamos el archivo JSON crudo que genera un usuario cada vez que da clic en un producto dentro de nuestra app de entregas, con todo y errores de red.
- **Silver:** Limpiamos ese JSON; filtramos los clics repetidos por error del sistema, validamos que el ID del usuario exista y lo guardamos en una tabla estructurada y limpia de "Interacciones de Usuarios".
- **Gold:** Contamos cuántos clics totales acumuló cada categoría de producto en las últimas 24 horas y calculamos el KPI de "Productos más Tendencia" para que el gerente decida qué promociones lanzar hoy mismo.



Gobernanza NoSQL y Teorema de CAP

Al diseñar sistemas distribuidos, se debe considerar el Teorema de CAP, que establece que solo se pueden garantizar dos de tres propiedades ante un fallo de red: **Consistencia** (C), **Disponibilidad** (A) o Tolerancia a **Particiones** (P).

- **Bases Documentales (MongoDB):** Priorizan la consistencia (CP), suspendiendo operaciones si hay riesgo de desfase de datos.
- **Orientadas a Columnas (Cassandra):** Priorizan la disponibilidad (AP), devolviendo respuestas continuas mediante consistencia eventual.

Ejemplo: E-commerce

Si estamos diseñando el almacenamiento para las transacciones de dinero, el saldo de las cuentas o la validación de IDs de usuario en la capa Silver, **no podemos permitirnos errores**. Si el servidor de Arequipa se desconecta por un milisegundo del servidor de Lima, prefiero bloquear temporalmente la operación antes de arriesgarme a que un usuario gaste el mismo dinero dos veces.

Aquí elegimos una base de datos **Documental como MongoDB**, que es de tipo **CP**. Prefiere suspender o pausar las operaciones si hay riesgo de desfasar o duplicar transacciones de dinero, asegurando precisión matemática.

Ejemplo: E-commerce

Ahora piensen en el gerente de marketing mirando el KPI de 'Productos más Tendencia' en la capa Gold para lanzar sus cupones. Si hay un micro-corte de red entre los servidores, ¿vale la pena colgar o congelar la app del gerente solo para esperar a que el último clic se sincronice perfectamente? ¡Por supuesto que no! Al gerente le sirve ver el reporte de inmediato, aunque tenga un desfase insignificante de unos cuantos segundos.

Para esta capa analítica o para el catálogo de productos de la app, elegimos una base de datos **Orientada a Columnas como Apache Cassandra**, que es de tipo **AP**. Cassandra prioriza dar una respuesta continua y ultra veloz al usuario. Garantiza lo que llamamos **Consistencia Eventual**: el sistema no se detiene, y en cuestión de milisegundos o segundos, todos los servidores se terminarán de poner al día de forma transparente.

contenido

1

Tema 1

2

Tema 2

3

Tema 3

4

Open data y su impacto en el
análisis de negocios.

Open Data

Open Data se refiere a conjuntos de datos accesibles, reutilizables y compartibles por cualquier persona sin barreras restrictivas. Se define por tres criterios fundamentales:

- Debe ser accesible (disponible en línea de forma gratuita).
- Permitir la reutilización y redistribución (bajo licencias que autoricen mezclarlo con otros conjuntos de datos).
- Garantizar la participación universal (sin discriminación de uso).

A diferencia de los datos privados de una empresa, el Open Data público es un mandato regulatorio para asegurar la transparencia y la rendición de cuentas.

¿Dónde conseguir datos abiertos?

Existen diversos portales y herramientas para acceder a estos activos de información:

- **España:** El portal nacional datos.gob.es ofrece catálogos de datos sobre economía, transporte y medio ambiente, facilitando soluciones para empresas españolas como Atogu (prospección B2B) o Iternova (mantenimiento de carreteras).
- **Estados Unidos:** El portal data.gov aloja más de 250,000 conjuntos de datos públicos útiles para regulación financiera, salud y desarrollo de negocios.
- **Market Intelligence API (ITA):** Ofrece reportes detallados y metadatos sobre oportunidades comerciales por país.

Impacto en la Competitividad y Democratización

Equilibrio de Mercado: Los portales gubernamentales actúan como un contrapeso económico, permitiendo que pymes y startups accedan gratuitamente a inteligencia de mercado (datos demográficos, catastrales o geográficos) que antes era monopolio de grandes corporaciones con presupuestos masivos.

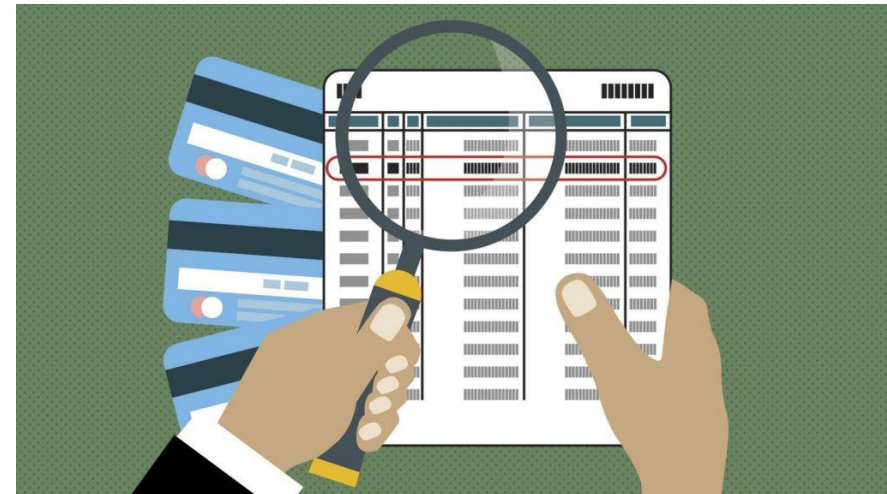
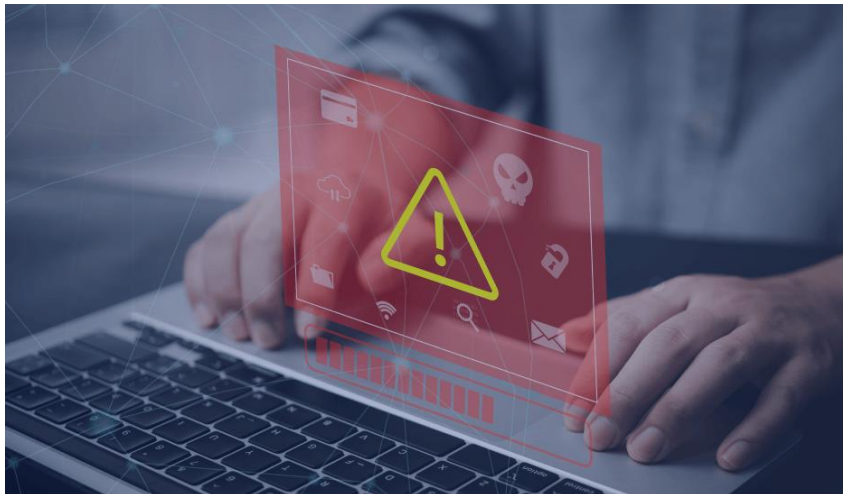
Reducción de la Asimetría de Información: El acceso a los mismos datasets que las agencias públicas permite a las empresas identificar nichos de mercado y vacíos de oferta antes de invertir en infraestructura.



Impacto Financiero y Social Demostrado

Detección de Fraude: En la Unión Europea, el uso de datos abiertos ha prevenido pérdidas de 2,500 millones de euros en proyectos financiados; en Brasil, el Portal de Transparencia identificó fraudes por 1,000 millones de dólares.

Transparencia Económica: La integración de IA y Open Data permite monitorear hasta 3 millones de transacciones diarias en países como Singapur para detectar colusión en licitaciones estatales.



Casos de Éxito: IterNova

IterNova es una destacada empresa tecnológica española que se ha convertido en un referente internacional al demostrar cómo los Datos Abiertos (Open Data) pueden transformarse en un modelo de negocio comercial altamente rentable y con un gran impacto social. La compañía identificó que los gobiernos locales y las empresas concesionarias gastan millones de euros en mantener infraestructuras como carreteras y puentes, pero casi siempre lo hacen de forma reactiva, es decir, reparando el daño cuando el bache o la grieta ya aparecieron.



Metodología para Inteligencia de Negocios

Para convertir estos datos rígidos en valor, las áreas de BI aplican:

Modelado de Temas (LDA): Algoritmos que extraen automáticamente tendencias e intereses de documentos legales no estructurados.

Visualización Semántica: Herramientas como que permiten a los analistas identificar prioridades de inversión pública de forma intuitiva.

Conexión vía APIs: El uso de APIs de Inteligencia de Mercado (como las del Departamento de Comercio de EE.UU.) facilita la ingesta de datos "vivos" para actualizar proyecciones de exportación/importación en tiempo real.

Trabajo Práctico

En esta fase, el estudiante explorará la arquitectura, el procesamiento y el enriquecimiento con datos externos.

Uso de Datos Abiertos (Open Data): Identifiquen fuentes externas que mitiguen la asimetría de información de su empresa:

- Localizar al menos un conjunto de datos en portales como datos.gob.es, el Portal de la UE, el Banco Mundial u otros que complementen su análisis.
- Explicar cómo este dato externo (ej. clima, tráfico o demografía) reduce el sesgo en sus proyecciones de negocio.

Trabajo Práctico

Propuesta de Arquitectura de Procesamiento

Basándose en los desafíos de **Velocidad** y **Volumen** que describieron en la fase anterior, definan la infraestructura técnica:

- **Hadoop vs. Spark:** Deben elegir y justificar qué motor usarán. ¿Necesitan Apache **Hadoop** para procesamiento masivo por lotes a bajo costo o Apache **Spark** para análisis en tiempo real en memoria?.
- **Arquitectura de Flujo:** Deben decidir entre una Arquitectura **Lambda** (si requieren máxima precisión histórica y datos rápidos por separado) o una Arquitectura **Kappa** (si prefieren un flujo único basado en streaming como **Kafka**).

Trabajo Práctico

Diseño del Pipeline y Almacenamiento: Esquematizar su flujo:

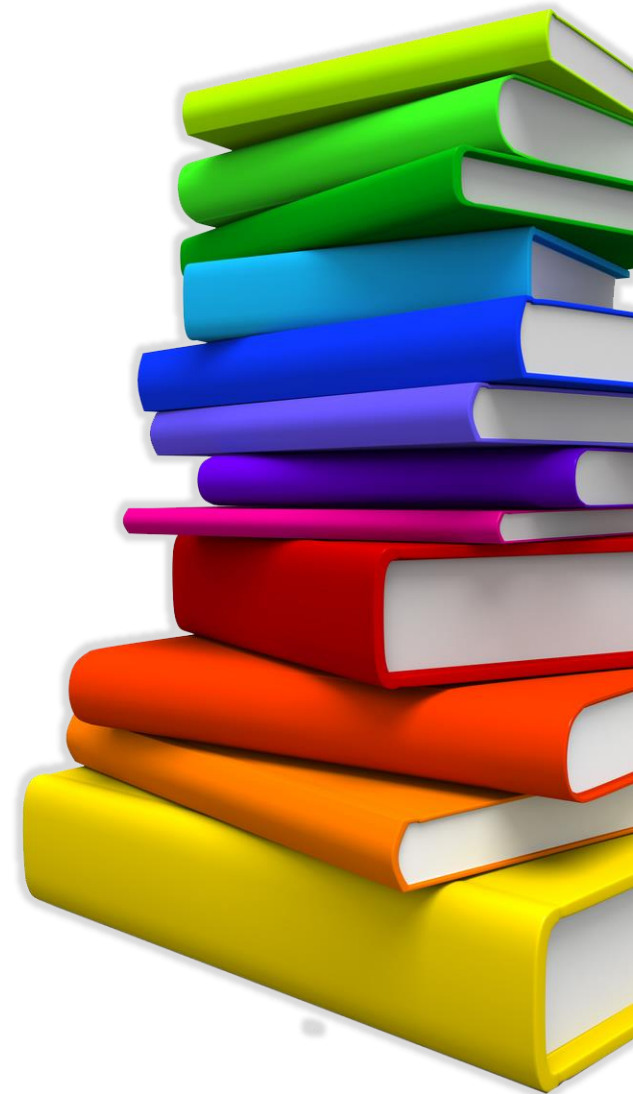
- **Arquitectura de Medallón:** Definir qué ocurrirá en cada capa:
 - **Capa Bronze:** Ingesta de datos crudos.
 - **Capa Silver:** Limpieza, filtrado y normalización.
 - **Capa Gold:** Agregaciones finales listas para el negocio.
- **Estrategia NoSQL:** Si su problema incluye datos no estructurados, deben elegir un motor bajo el **Teorema de CAP**. Por ejemplo, MongoDB si priorizan la consistencia o Cassandra si priorizan la disponibilidad.

Conclusiones

- Manejar volúmenes a escala de Petabytes obliga a migrar a un escalado horizontal, donde Hadoop asegura el almacenamiento histórico económico en disco y Spark tritura los datos en tiempo real dentro de la memoria RAM.
- Mientras la arquitectura Lambda divide el procesamiento en dos vías independientes (lotes y streaming) duplicando el código, la arquitectura Kappa unifica todo en un solo motor continuo sobre una cola de datos inmutable, permitiendo recalcular el pasado con solo rebobinar el tiempo.
- Los datos refinados secuencialmente mediante la arquitectura Medallion deben persistir bajo las reglas de CAP; ante un fallo de red inevitable, la ingeniería obliga a elegir entre Consistencia absoluta o Disponibilidad total.

Bibliografía

- DAMA International. (2017). DAMA-DMBOK: Guía del cuerpo de conocimiento de la gestión de datos (2da ed.). Technics Publications.
- Han, J., Kamber, M., y Pei, J. (2012). Data mining: Concepts and techniques [Minería de datos: Conceptos y técnicas] (3ra ed.). Morgan Kaufmann.
- Kimball, R., y Ross, M. (2013). The data warehouse toolkit: The definitive guide to dimensional modeling [El kit de herramientas del almacén de datos: La guía definitiva para el modelado dimensional] (3ra ed.). John Wiley & Sons.
- Montgomery, D. C., y Runger, G. C. (2018). Applied statistics and probability for engineers [Estadística aplicada y probabilidad para ingenieros] (7ma ed.). John Wiley & Sons.
- Redman, T. C. (2016). Getting data quality right the first time [Hacer bien la calidad de los datos a la primera]. Harvard Business Review Press.





**Universidad
Tecnológica
del Perú**